

AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari



Panda\n



PANDAS

Derived from **PAN**el **DA**ta
a term for multidimensional structured datasets.

So, it's Not related to Panda :)

Standard Python is great, but it's vanilla. If you want to do high-level cognitive modeling or linguistic data wrangling, you need to call in the professionals.



Hello, future architect of the mind. If you are reading this, you've likely realized that standard Python is a bit like a basic survival kit: it has a knife and some matches, but you can't build a skyscraper with it. To do high-level research, we need specialized tools.

Imagine Python is a kitchen. By default, you have a stove and a fridge. That's your **Standard Library**. But if you want to make liquid nitrogen ice cream for a cognitive experiment, you need to order a specialized machine. In Python, that machine is a **Third-Party Library**.

How do you get it? You use **PIP (Pip Installs Packages)**. This is a program you run in your **CLI (Command Line Interface)**, that scary looking black box with white text, to download tools from the internet. Once downloaded, these tools live on your computer, ready for your next project.

```
import numpy as np # The math engine
import pandas as pd # The data wrangler
import sys # A standard library tool
```



Library	Name Origin	The "Under the Hood" Meaning
NumPy	N umerical P ython	The fundamental package for scientific computing; it replaces slow Python lists with high-speed, C-based numerical arrays.
Pandas	P anel D ata	Derived from econometrics, referring to multi-dimensional structured datasets. (It has nothing to do with the bamboo-eating bear)
Matplotlib	M ATLAB-style P lotting L ibrary	Created to mimic the graphing capabilities of MATLAB, providing a professional visualization engine for the Python ecosystem. (we will go for it on peacock session)



Let's start at the very beginning. You already know how to use a Python `List`. They are wonderful because they are easy, you can throw a word, a number, and even another list inside one. But in the world of LMT, easy often becomes slow. When Python manages a list, it creates 'pointers.' Think of this as a scavenger hunt: to find the second item in a list, Python has to look at the first item, find the address of the second item, and then jump to that location in your computer's RAM. This jumping around takes time.

Now, meet the **ndarray (N-Dimensional Array)**. This is the heart of NumPy. Unlike a list, `ndarray` is 'homogeneous', every single item inside it must be the same data type, like all integers or all floats. Because of this restriction, NumPy can store all that data in one solid, contiguous block of memory. Imagine you are a librarian. In a Python List, the books are scattered all over the building. In a NumPy Array, they are all on one shelf, in order. If I ask you to scan 10,000 books, which method is faster? This structural efficiency is why NumPy is the foundation of almost every scientific discovery made with Python today. It transforms your computer from a general-purpose machine into a specialized laboratory for high-speed data processing.



Feature	Python List	NumPy ndarray (N-Dimensional Array)	Technical Impact
Data Type	Heterogeneous (Can mix int, str, float)	Homogeneous (All elements must be the same type)	NumPy saves space by not "guessing" the type for every item.
Memory Layout	Scattered across RAM (Random Access Memory)	Contiguous (Stored in one solid, unbroken block)	Computers can read contiguous blocks much faster.
Scaling	Becomes very slow with millions of items.	Designed for Big Data (millions of data points)	Essential for processing large linguistic corpora or EEG signals.
Operations	Uses loops (slow)	Uses Vectorization (math on the whole block at once)	Reduces processing time from minutes to milliseconds.



Welcome back to the lab. Now that we know *why* we use **NumPy** (**N**umerical **P**ython), let's talk about how we build these structures. In your research, you rarely want to type out every single data point by hand.

First, we have **np.array()**. This is your bridge. It takes a standard Python list, maybe a list of word lengths you collected, and teleports it into the high-speed NumPy realm.

But what if you need to simulate 100 trials in a cognitive experiment? You use **np.arange()**. It's like an automated assembly line that stamps out numbers in order.

The most critical concept today, however, is **Shape**; we don't just look at words in a line; we look at structures. The **.shape** attribute tells you exactly how your data is organized. Is it a single line of 12 numbers? Or is it a 3x4 matrix representing 3 participants across 4 different linguistic tasks?

If the 'shape' isn't right, we use **.reshape()**. Think of this as digital origami. You aren't cutting the paper (the data stays the same), you are just folding it differently so the computer can 'see' the patterns you are looking for. It's the difference between looking at a pile of bricks and looking at a wall

ndarray



Component	Syntax	Technical Description
ndarray	<code>np.array([data])</code>	N-Dimensional Array (ndarray) ; a fast, space-efficient multidimensional array providing vectorized arithmetic operations.
Data Type	<code>array.dtype</code>	The fixed type of all elements in the array (e.g., int64, float32). Unlike lists, arrays are "homogeneous."
Shape	<code>array.shape</code>	A tuple representing the dimensions of the array (e.g., (rows, columns)).
Vectorization	<code>array * 2</code>	Performing a mathematical operation on the entire block of memory at once without using a Python for loop.

ndarray



```
import numpy as np

# 1. Standard List (Heterogeneous/Scattered)
counts_list = [100, 200, 300]

# 2. NumPy Array (Homogeneous/Contiguous)
counts_array = np.array([100, 200, 300])

# 3. Check the "Homogeneity" (The fixed type)
print(f>Data Type: {counts_array.dtype}")

# 4. Math without loops (Vectorization)
print(f>Fast Math: {counts_array * 2})
```

Numpy features



Component	Syntax	Technical Description
Array Creation	<code>np.array([1, 2, 3])</code>	Converts existing Python sequences (like lists) into a high-speed ndarray (N-Dimensional Array).
Range Generation	<code>np.arange(start, stop)</code>	Generates a sequence of numbers automatically. Similar to Python's <code>range()</code> but returns an array ready for math.
Shape Attribute	<code>array.shape</code>	An attribute (not a function) that returns a Tuple (a fixed list) representing the dimensions (rows, columns).
Reshaping	<code>array.reshape(x, y)</code>	Changes the 'perspective' of the data (e.g., turning a flat list of 12 numbers into a 3x4 grid) without changing the data itself.

Numpy features



```
import numpy as np

# 1. Automatic Creation (12 trial IDs)
trials = np.arange(1, 13)

# 2. Inspecting the 'Blueprint'
print(f"Original Shape: {trials.shape}") # Output: (12,)

# 3. The Digital Origami (3 participants, 4 tasks each)
# Note: 3 * 4 MUST equal 12!
matrix = trials.reshape(3, 4)

print("--- Reshaped Research Matrix ---")
print(matrix)
print(f"New Dimensions: {matrix.shape}")
```

Numpy features



Now we enter the realm of 'Vectorization.' If you remember one thing from today, let it be this: **Loops are the enemy of speed.**

In standard Python, if you want to double 1,000 reaction times, you write a loop that visits each number one by one. This is slow because the CPU has to stop and ask, 'What is this? Okay, it's an integer. Now double it. Next!'

Vectorization uses a trick called **SIMD** (**S**ingle **I**nstruction, **M**ultiple **D**ata). It tells the computer: 'Here is a block of 1,000 numbers. Double all of them *now*.' It happens in parallel. It's the difference between a hundred people standing in line for one coffee machine vs. a hundred coffee machines appearing at once.

We also have **Broadcasting**. This is NumPy's 'magic' logic. If you have a matrix of word frequencies and you want to subtract '1' from every single one, you don't need a matrix of ones. NumPy just 'broadcasts' that single number across the whole structure.

Finally, for those of you interested in Stochastic (Probabilistic) modeling, like predicting which word a person will say next, we use the **PRNG** (**P**seudo-**R**andom **N**umber **G**enerator). By setting a **Seed**, you ensure that the 'random' noise in your experiment is the same for you as it is for a reviewer in another country. This is the cornerstone of scientific reproducibility.

Numpy features



Component	Syntax	Technical Description
Vectorization	<code>array * 2</code>	Performing an operation on all elements of an ndarray (N-Dimensional Array) simultaneously at the CPU (Central Processing Unit) level.
Broadcasting	<code>array + scalar</code>	A NumPy rule that allows math between arrays of different shapes (e.g., adding a single number to a whole grid) by "stretching" the smaller value.
PRNG	<code>np.random.rand()</code>	Pseudo-Random Number Generator ; creates arrays of random values used for stochastic (probabilistic) modeling in linguistics.
Seed	<code>np.random.seed(42)</code>	A starting point for the PRNG that ensures your "random" data is reproducible for other researchers.

Numpy features



```
import numpy as np

# 1. Vectorized Math (No loops!)
scores = np.array([80, 90, 70])
normalized = (scores - 50) / 10 # Broadcasting in action
print(f"Standardized Scores: {normalized}")

# 2. Reproducible Randomness
np.random.seed(2026) # The 'Lab Key'
noise = np.random.normal(0, 1, (3, 3)) # 3x3 grid of 'brain noise'
print(f"Stochastic Sample:\n{noise}")
```

Numpy features



Now that we have built our high-speed **ndarray** and filled it with data, we need to make sense of it. In the interdisciplinary world of LMT, we rarely care about a single reaction time; we care about the average reaction time across a thousand trials.

This is where **NumPy** shines with **Statistical Aggregations**. These are functions that take a massive amount of data and 'collapse' it into a single, meaningful number, like a mean or a sum. However, when our data becomes a matrix (rows and columns), we encounter the concept of the **Axis**. This is often the most confusing part for new researchers. Think of it this way: if you have a table where each row is a 'Participant' and each column is a 'Task,' you have two ways to look at the data.

If you want the average score for 'Task A,' you need to look *down* the column, this is **Axis 0**. You are collapsing all the rows into one. If you want the average score for 'Participant 1' across all their tasks, you look *across* the row, this is **Axis 1**. You are collapsing the columns. Mastering these axes is the difference between a researcher who understands their data and one who is just pressing buttons in the dark.

Numpy features



Component	Syntax	Technical Description
Mean	<code>np.mean(array)</code>	Calculates the arithmetic average; the most common measure of "central tendency" in behavioral data.
Standard Deviation	<code>np.std(array)</code>	Std andard Dev iation (std); quantifies the amount of variation or dispersion in a set of values.
Sum	<code>np.sum(array)</code>	The total value of all elements; useful for calculating total word counts or aggregate scores.
Axis	<code>axis=0</code> or <code>axis=1</code>	The direction of the calculation: 0 collapses rows (result per column), 1 collapses columns (result per row).

Numpy features



```
import numpy as np

# 2 Participants, 3 Word-Recall Tasks
# Rows = Participants | Columns = Tasks
results = np.array([[15, 20, 18],
                    [12, 25, 22]])

# 1. Total average (the whole experiment)
print(f"Overall Mean: {np.mean(results):.2f}")

# 2. Average per Task (Look down: Axis 0)
print(f"Mean per Task: {np.mean(results, axis=0)}")

# 3. Variability per Participant (Look across: Axis 1)
print(f"SD per Participant: {np.std(results, axis=1)}")
```



We've mastered **NumPy** (**N**umerical **P**ython), but let's be honest: looking at a raw array of numbers is like trying to read a book written in binary. We need context. We need **Pandas** (**PAN**el **DA**ta). Think of a **Series** as a single column of your research data, perhaps a list of scores for a single phoneme recognition task. It has the speed of a NumPy array, but it adds an **Index**. This index is a set of labels that tell you exactly *who* or *what* each number belongs to.

When you stack multiple Series together, you get a **DataFrame**. This is the digital equivalent of a laboratory notebook or a CSV file. Your DataFrame might have columns for 'Participant_ID', 'Native_Language', and 'Response_Accuracy'.

The beauty of the DataFrame is that it handles 'heterogeneous' data. Unlike NumPy, which demands everyone look the same, a DataFrame is perfectly happy having a column of names (strings) sitting right next to a column of ages (integers). It is the ultimate tool for organizing the messy, diverse reality of human linguistic behavior.

Dataframe



Component	Syntax	Technical Description
Series	<code>pd.Series(data)</code>	A one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects). Think of it as a single column in a spreadsheet.
DataFrame	<code>pd.DataFrame(data)</code>	A two-dimensional, size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). The "bread and butter" of data science.
Index	<code>df.index</code>	The "address" system of the data. It labels the rows, allowing you to find a specific participant's data instantly without counting rows.
Columns	<code>df.columns</code>	The labels for the vertical axes, representing different variables or features in your research (e.g., 'Reaction_Time', 'Word_Type').

Dataframe



```
import pandas as pd

# 1. The Series (A single column of 'Word Lengths')
lengths = pd.Series([5, 3, 8], index=['Apple', 'Cat', 'Elephant'])

# 2. The DataFrame (The full Lab Table)
data = {
    'Word': ['Apple', 'Cat', 'Elephant'],
    'Type': ['Fruit', 'Animal', 'Animal'],
    'Time_ms': [450, 320, 710]
}
df = pd.DataFrame(data)

print("--- Lab Series ---")
print(lengths)
print("\n--- Research DataFrame ---")
print(df)
```



Now that we understand the anatomy of a DataFrame, we need to get some actual research data into it. You will rarely type your data manually. Instead, you will receive files from an EEG machine, an eye-tracker, or an online survey.

The most common format is the CSV. It is the simplest way to store a table: it's just a text file where a comma acts as a 'wall' between columns. When we use **pd.read_csv()**, Pandas acts as an automated translator. It reads that text, recognizes the 'walls,' and builds a structured DataFrame in your computer's memory.

But beware, not all 'walls' are commas! In some European countries, they use semicolons (;) because they use commas for decimal points. If your data looks like a giant, messy sentence when you import it, you simply tell Pandas: **sep=';'**

Once you've finished your Data Wrangling, cleaning up the messy bits, you need to save your work. Using **.to_csv()** 'freezes' your digital laboratory into a physical file you can send to a supervisor or include in a publication. It is the final step in the automation of behavioral datasets.

Data Ingestion



Component	Syntax	Technical Description
Read CSV	<code>pd.read_csv('file.csv')</code>	Loads a CSV file from your computer and converts it into a DataFrame.
To CSV	<code>df.to_csv('new.csv')</code>	Exports the current DataFrame back into a physical file on your hard drive.
Separator	<code>sep=';'</code>	A parameter used to tell Pandas which character (comma, semicolon, tab) is used to divide the data columns.
Index Parameter	<code>index=False</code>	An argument used during export to prevent Pandas from saving the row numbers as a new column.



```
import pandas as pd

# 1. Importing the 'Raw' Research Data
# We assume 'transcripts.csv' is in the same folder as our script
df = pd.read_csv('transcripts.csv')

# 2. Inspecting the first 5 rows to ensure it loaded correctly
print(df.head())

# 3. Exporting the 'Clean' version
# index=False ensures we don't save the row numbers (0, 1, 2...)
df.to_csv('cleaned_results.csv', index=False)
```



EDA (Exploratory **D**ata **A**nalysis), Think of this as a 'Clinical Checkup' for your dataset.

We start with **.head()**. This is like glancing at the first page of a book to make sure it's written in a language you understand. It shows you the top 5 rows by default. If your column names look like gibberish here, you know you have an import problem immediately.

Next is **.info()**. This is the skeleton of your research. It tells you if your 'Reaction Time' column is actually being treated as a number (float) or if Python thinks it's a word (object). It also reveals the **Invisible Gaps**, missing values that could ruin your statistics later.

Then we have **.describe()**. This gives you the 'Pulse' of the data. If you are studying word lengths and **.describe()** tells you the maximum length is 400 letters, you don't have a word; you have a paragraph that accidentally ended up in the wrong column.

Finally, **.value_counts()**. We often deal with categories like 'Native Speaker' vs. 'Non-Native.' This method tells you if your groups are balanced. If you have 99 native speakers and only 1 non-native, your comparative study is over before it began. Always audit before you analyze.



Component	Syntax	Technical Description
Read CSV	<code>pd.read_csv('file.csv')</code>	Loads a CSV file from your computer and converts it into a DataFrame.
To CSV	<code>df.to_csv('new.csv')</code>	Exports the current DataFrame back into a physical file on your hard drive.
Separator	<code>sep=';'</code>	A parameter used to tell Pandas which character (comma, semicolon, tab) is used to divide the data columns.
Index Parameter	<code>index=False</code>	An argument used during export to prevent Pandas from saving the row numbers as a new column.



```
import pandas as pd

# 1. Loading the Research Data
df = pd.read_csv('session6_data.csv')

# 2. The 'Quick Glance'
print(df.head(3))

# 3. The 'Skeleton Check'
df.info()

# 4. The 'Statistical Pulse'
print(df.describe())

# 5. The 'Group Balance'
print(df['Participant_Group'].value_counts())
```



There are two primary ways to 'address' your data in Pandas : **.loc** and **.iloc**.

Think of **.loc** as a street address. If I want to find the reaction time for 'Participant_42', I use `.loc['Participant_42', 'Time_ms']`. It is semantic and descriptive. It survives even if you shuffle the rows because the 'name' of the participant stays the same.

Think of **.iloc** as a GPS coordinate. It doesn't care about names; it only cares about the physical layout of the table. `.iloc[0, 2]` means 'give me whatever is in the first row and the third column.' This is incredibly powerful when you are writing automated scripts that need to process 'the last five participants' regardless of who they are.

The trickiest part for new researchers is **Slicing**. When you slice with numbers in Python (like `.iloc[0:5]`), you get rows 0 through 4. But when you slice with labels (like `.loc['A':'C']`), you get A, B, *and* C. It's a small detail, but it's the difference between including your best data point and accidentally deleting it.



Component	Syntax	Technical Description
.loc	<code>df.loc[label]</code>	Location -based selection; used to find data using the explicit names of rows or columns (e.g., 'Participant_01').
.iloc	<code>df.iloc[integer]</code>	Integer- Location selection; used to find data by its numerical position in the table (e.g., row 0, column 2).
Slicing	<code>df.loc[start:stop]</code>	Selecting a range of data points simultaneously; note that .loc is inclusive of the 'stop' label, while .iloc is exclusive.
Column Access	<code>df['ColumnName']</code>	The standard way to isolate a single variable (a Series) from the DataFrame.



```
import pandas as pd

# Creating a small linguistic lab table
df = pd.DataFrame({
    'Word': ['Apple', 'Idea', 'Run'],
    'Type': ['Noun', 'Abstract', 'Verb']
}, index=['ID1', 'ID2', 'ID3'])

# 1. Access by Label (Street Name)
print(df.loc['ID2', 'Word']) # Output: Idea

# 2. Access by Position (GPS Coordinate)
print(df.iloc[0, 0])      # Output: Apple

# 3. Accessing a whole column
print(df['Type'])
```



raw data is often 'ugly.' Your eye-tracker might export a column called `V1_RT_final_revised_2`, which is a nightmare to type. Or perhaps you've realized that you need a new column that converts 'Milliseconds' into 'Seconds.' This is where we perform **Structural Edits**. Think of this as the 'remodeling phase' of your digital laboratory.

First, we have **Column Assignment**. This is how we grow our dataset. If you have a column of reaction times, you can simply divide it by 1000 and assign it to a new name. Pandas will automatically 'broadcast' that math across every single row.

Next is **`df.rename()`**. This is about clarity. In a collaborative environment, clear variable names are a form of empathy for your future self (and your co-authors).

Finally, we have **`df.drop()`**. Science is as much about what you ignore as what you measure. If your CSV file contains technical metadata that doesn't help answer your linguistic hypothesis, get rid of it. A clean lab is a fast lab. Just remember when you drop a column, it's gone from the current IDE session unless you reload the file. Measure twice, cut once.



Component	Syntax	Technical Description
Column Assignment	<code>df['New_Col'] = data</code>	Creates a brand-new variable in your DataFrame (DF) by assigning a list, array, or calculation to a new label.
Renaming	<code>df.rename(columns={'Old':'New'})</code>	Changes specific column headers; essential for transforming cryptic machine output into human-readable research variables.
Dropping	<code>df.drop(columns=['ColName'])</code>	Removes unnecessary data to reduce the RAM (Random Access Memory) footprint and clear mental clutter.
Inplace Parameter	<code>inplace=True</code>	A setting that modifies the original object directly rather than creating a new copy; use with caution in the 2026 research environment.



```
import pandas as pd

# Initial Lab Data
df = pd.DataFrame({'RT': [450, 520], 'Error': [0, 1]})

# 1. Renaming for clarity
df = df.rename(columns={'RT': 'Reaction_Time_ms'})

# 2. Adding a calculated column (Vectorization)
df['In_Seconds'] = df['Reaction_Time_ms'] / 1000

# 3. Dropping the original 'messy' column
# We keep only the polished variables
df_clean = df.drop(columns=['Reaction_Time_ms'])

print("--- Remodeled DataFrame ---")
print(df_clean)
```



Garbage in, garbage out. If your dataset is full of duplicates or your RT (Reaction Time) columns are accidentally formatted as text, your statistical models will be useless.

We begin with **Boolean Logic**. Think of this as the 'Gatekeeper' of your lab. Using operators like **AND** (&), **OR** (|), and **NOT** (~), you can create complex rules. For instance: Give me all participants who are over 20 years old **AND** did not fail the attention check.

But what if the data is just... missing? Perhaps an eye-tracker lost its connection for a second. This creates a **NaN** (**N**ot **a** **N**umber). You have two choices: delete the row using **.dropna()** or perform **Imputation** using **.fillna()** Imputation is the act of 'guessing' the missing value, perhaps by using the average score of the group. It's a delicate scientific choice that requires a justification in your methodology section.

Finally, we have **Casting** via **.astype()** Computers are literal. If a participant accidentally typed '25' instead of just 25, Python might treat that whole column as text. You can't calculate the 'mean' of a word. By 'casting' that column to an integer or float, you are telling the computer: 'Trust me, this is a number.'



Method	Syntax	Technical Description
Boolean Logic	& (AND), (OR) ~ (NOT)	Multi-condition filtering used to isolate specific subsets of research data (e.g., only native speakers who passed the test).
Pattern Matching	.isin(), .str.contains()	Targeted selection based on lists of categories or specific text substrings within strings.
Data Integrity	.isna(), .dropna(), .drop_duplicates()	Tools for detecting and removing NaN (Not a Number) values or redundant participant entries.
Imputation & Casting	.fillna(), .astype()	Filling in "holes" in the dataset and forcing columns into the correct format (e.g., turning text ages into numbers).



```
import pandas as pd
import numpy as np
# A typical messy linguistic dataset
df = pd.DataFrame({
    'Subject': [1, 2, 2, 3], # Subject 2 is a duplicate!
    'Group': ['Native', 'L2', 'L2', 'L2'],
    'Score': [95, np.nan, 88, 72] # Subject 2 has a missing score
})
# 1. Drop duplicates and handle missingness
df = df.drop_duplicates()
df['Score'] = df['Score'].fillna(df['Score'].mean()) # Impute with average

# 2. Multi-condition Filter: Native speakers OR high scorers
# Note: Use () around each condition!
top_tier = df[(df['Group'] == 'Native') | (df['Score'] > 90)]

# 3. Correct data types (Force Subject to be a string label)
df['Subject'] = df['Subject'].astype(str)

print("--- Cleaned Research Data ---")
print(df)
```


Quiz



I am uploading two files to Discord right now: a 100-row CSV file and a PDF containing your protocol manual. We call this **The Polyglot's Sabotage**.

This is a **Double-Quiz Event**. Your code will be graded as **Quiz 03**, focusing on your **Code Integrity**, how clean, efficient, and logical your solution is. Your final statistical results, the Mean and Standard Deviation, will be graded as **Quiz 04**, focusing on your **Analytic Insight**.

To succeed, you must be a forensic linguist. Follow the four protocols in the PDF, Selection, Standardization, Recovery, and Exclusion, to the letter. Once you've filtered the noise and found the results, DM me your code and your results. The digital lab is now open.

Good luck, architects.